

RAG Pipeline — LangChain and ChromaDB

Retrieval-Augmented Generation, commonly called RAG, combines information retrieval with text generation. A RAG system first retrieves relevant information from a knowledge base and then provides that information to a language model as context.

Embeddings are numerical vector representations of text. They capture semantic meaning, allowing text with similar meanings to be located close together in vector space.

ChromaDB is a vector database used to store embeddings and perform similarity searches. It can also store metadata associated with documents.

LangChain provides components for building applications with language models. These components include document loaders, text splitters, embeddings, retrievers, vector stores, prompts, and language models.

Ollama allows large language models and embedding models to run locally. It can be used to build local RAG applications without requiring an external cloud API for the model inference.

A typical RAG pipeline consists of loading documents, splitting documents into chunks, generating embeddings, storing the embeddings in a vector database, retrieving the most relevant chunks for a question, and passing the retrieved context to a language model.

ChromaDB can perform similarity search using vector embeddings. Cosine similarity is commonly used with text embeddings because it measures the similarity between vectors based on the angle between them.

In this practical project, LangChain, ChromaDB, and Ollama are combined to create a local RAG pipeline. The PDF is loaded and split into chunks, the chunks are converted into embeddings using the Ollama embedding model, and the embeddings are stored in ChromaDB. When a question is asked, the three most relevant chunks are retrieved and passed to the Ollama language model to generate an answer.